

GDG Workshop Backend

Project Deep Dive — Complete Reference

github.com/sujallchaudhary/gdg-workshop-backend

1. What is This Project?

This is a Node.js + Express backend that takes a user's game idea (a text prompt) and uses an LLM to generate a complete, playable 2D browser game — fully working HTML, CSS, and JavaScript code — along with an AI-generated thumbnail image, all in one API call flow.

It is not just a chatbot. It is a full async task pipeline that:

- Accepts a user's game prompt
- Spins up a background task (so the user does not wait)
- Calls the Nebius LLM API to generate game code (HTML + CSS + JS)
- Generates a thumbnail image for the game
- Saves everything to MongoDB
- Lets users poll for the result and view completed games

 *End Result: User types 'Create a snake game' → backend generates a fully playable snake game in the browser with a thumbnail. No human writes the game code. The LLM does.*

1.1 Tech Stack

Runtime	Node.js
Framework	Express.js
LLM Provider	Nebius AI (OpenAI-compatible API)
Database	MongoDB (via Mongoose)
Task Pattern	Async background task with UUID-based task IDs
Image Gen	LLM-based thumbnail generation (via Nebius)
Frontend	Static HTML/CSS/JS served from public/ folder
Config	.env for secrets (NEBIUS_API_KEY, MONGODB_URI, PORT)

2. Exact Project Folder Structure

This is the actual structure of the repo — every folder and file:

```
gdg-workshop-backend/
├── index.js                ← Entry point. Sets up Express, DB, routes, server
├── routes/
│   └── taskRoutes.js      ← Defines POST /api/tasks and GET
                             /api/tasks/:taskId
                             and GET /api/games
├── controllers/
│   └── taskController.js  ← Handles req/res. Calls service. Sends JSON back.
├── service/
│   └── taskService.js     ← Core business logic. Orchestrates the full
                             LLM game generation pipeline.
├── chains/
│   ├── gameChain.js      ← LLM call for generating HTML+CSS+JS game code
│   └── imageChain.js      ← LLM call for generating the game thumbnail
├── llm/
│   └── nebius.js          ← Nebius API client setup (OpenAI-compatible)
├── prompt/
│   ├── gamePrompt.js     ← System + user prompt template for game code gen
│   └── imagePrompt.js     ← Prompt template for thumbnail generation
├── models/
│   └── Task.js            ← Mongoose schema: taskId, status, title,
                             html, css, js, thumbnail, createdAt
├── middlewares/
│   └── errorHandler.js    ← Global Express error handling middleware
├── public/
│   └── index.html         ← Frontend UI served as static file
├── .env.example           ← Template showing required env variables
├── .gitignore             ← Ignores node_modules, .env
├── package.json           ← Project manifest and dependencies
└── README.md              ← Setup instructions
```

3. API Endpoints — What the Backend Exposes

This backend exposes three REST API endpoints:

POST /api/tasks	Submit a game generation task. Body: { prompt }. Returns: { taskId }
GET /api/tasks/:taskId	Poll for task status. Returns status + full game data when complete.
GET /api/games	List all completed games (title, thumbnail, taskId, createdAt only)

3.1 POST /api/tasks — Submit a Game

Request

```
POST http://your-server/api/tasks
Content-Type: application/json

{
  "prompt": "Create a snake game with neon colors"
}
```

Immediate Response (task queued)

```
{
  "taskId": "a3f2c1d4-5678-4abc-9def-0123456789ab"
}
```

Notice: the server returns immediately with a taskId. It does NOT wait for the game to be generated. This is the async task pattern — generation happens in the background.

3.2 GET /api/tasks/:taskId — Poll for Result

While processing

```
{
  "taskId": "a3f2c1d4-...",
  "status": "processing"
}
```

When completed

```
{
  "taskId": "a3f2c1d4-...",
  "status": "completed",
  "title": "Neon Snake Game",
  "html": "<!DOCTYPE html>...",
}
```

```
"css": "body { background: #000; } ...",  
"js": "const canvas = document.getElementById(...",  
"thumbnail": "data:image/png;base64,..."  
}
```

3.3 GET /api/games — List All Games

```
{  
  "games": [  
    {  
      "taskId": "a3f2c1d4-...",  
      "title": "Neon Snake Game",  
      "thumbnail": "data:image/png;base64,...",  
      "createdAt": "2024-01-15T10:30:00Z"  
    },  
    { ... }  
  ]  
}
```

💡 Why only titles in GET /api/games? The html/css/js fields are huge. Returning them for every game in a list would be slow and wasteful. Fetch full data only for one game at a time via GET /api/tasks/:taskId.

4. Game Generation Flow — Step by Step

This is the most important part to understand. Here is exactly what happens from the moment a user submits a prompt to the moment they can play the game:

1. User submits POST /api/tasks with their game prompt
2. Controller receives the request, calls taskService.createTask(prompt)
3. Service generates a UUID as the taskId
4. Service creates a MongoDB document: { taskId, status: 'processing', prompt }
5. Service responds immediately — returns the taskId to the controller
6. Controller sends { taskId } back to the user — request ends here
7. Meanwhile, in the background, the service starts the generation pipeline:
8. → gameChain.js is called: sends prompt to Nebius LLM → gets back HTML + CSS + JS
9. → imageChain.js is called: generates a base64 thumbnail image for the game
10. Service updates the MongoDB document with the generated code + thumbnail + status: 'completed'
11. User polls GET /api/tasks/:taskId and eventually gets the full game data

 **Key Architecture Decision:** Why async? Game generation via LLM can take 10-30 seconds. If we made the user wait, the HTTP request would time out and the experience would be terrible. By returning a taskId immediately and having the user poll, the UX is smooth.

4.1 Visual Flow Diagram

Step 1	User → POST /api/tasks { prompt: '...' }
Step 2	Controller → taskService.createTask(prompt)
Step 3	Service → MongoDB.create({ taskId, status: 'processing' })
Step 4	Service → returns taskId immediately
Step 5	Controller → res.json({ taskId }) ← HTTP response ends
Step 6 (BG)	Service → gameChain(prompt) → Nebius API → { html, css, js }
Step 7 (BG)	Service → imageChain(title) → Nebius API → base64 thumbnail
Step 8 (BG)	Service → MongoDB.update({ status: 'completed', html, css, js, thumbnail })
Step 9	User polls GET /api/tasks/:taskId → gets completed game

5. File-by-File Walkthrough

5.1 index.js — The Entry Point

This is where your application starts. It wires everything together.

What it does:

- Loads environment variables via dotenv
- Creates the Express app
- Connects to MongoDB using Mongoose
- Registers middleware (express.json() to parse request bodies)
- Mounts the task routes at /api
- Registers the global error handler middleware (last)
- Starts the HTTP server on process.env.PORT || 3000

```
// Rough structure of index.js
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const taskRoutes = require('./routes/taskRoutes');
const errorHandler = require('./middlewares/errorHandler');

const app = express();
app.use(express.json());
app.use(express.static('public')); // serve frontend

mongoose.connect(process.env.MONGODB_URI);

app.use('/api', taskRoutes);
app.use(errorHandler); // must be last

app.listen(process.env.PORT || 3000);
```

5.2 routes/taskRoutes.js — URL Definitions

Routes file only defines WHAT URLs exist and WHICH controller handles them. No logic here.

```
const express = require('express');
const router = express.Router();
const taskController = require('../controllers/taskController');

router.post('/tasks', taskController.createTask);
router.get('/tasks/:taskId', taskController.getTask);
router.get('/games', taskController.getAllGames);
```

```
module.exports = router;
```

5.3 controllers/taskController.js — Request & Response

Controllers handle the HTTP layer only. They read from req, call the service, and write to res.

```
exports.createTask = async (req, res, next) => {
  try {
    const { prompt } = req.body;
    const taskId = await taskService.createTask(prompt);
    res.json({ taskId });
  } catch (err) { next(err); }
};

exports.getTask = async (req, res, next) => {
  try {
    const task = await taskService.getTask(req.params.taskId);
    res.json(task);
  } catch (err) { next(err); }
};

exports.getAllGames = async (req, res, next) => {
  try {
    const games = await taskService.getAllGames();
    res.json({ games });
  } catch (err) { next(err); }
};
```

 Notice `next(err)`: Instead of handling errors inline, controllers pass errors to `next()`. Express then forwards them to the global `ErrorHandler` middleware. This keeps controllers clean.

5.4 service/taskService.js — The Core Logic

This is the most important file. It orchestrates everything: creates the task, triggers background generation, updates the database.

Key responsibilities:

- Generate a unique `taskId` (using `uuid` or `crypto.randomUUID()`)
- Create a MongoDB Task document with status: 'processing'
- Return the `taskId` immediately (so controller can respond)
- Start async background generation (NOT awaited at response time)
- Call `gameChain` to get HTML + CSS + JS from LLM
- Call `imageChain` to get the thumbnail
- Update MongoDB Task document to status: 'completed' with all data

- Handle errors — if generation fails, update status to 'failed'

```
// Simplified structure
const { v4: uuidv4 } = require('uuid');
const Task = require('../models/Task');
const { generateGame } = require('../chains/gameChain');
const { generateImage } = require('../chains/imageChain');

exports.createTask = async (prompt) => {
  const taskId = uuidv4();
  await Task.create({ taskId, status: 'processing', prompt });

  // Fire and forget — not awaited
  generateGameInBackground(taskId, prompt);

  return taskId;
};

async function generateGameInBackground(taskId, prompt) {
  try {
    const { title, html, css, js } = await generateGame(prompt);
    const thumbnail = await generateImage(title);
    await Task.findOneAndUpdate(
      { taskId },
      { status: 'completed', title, html, css, js, thumbnail }
    );
  } catch (err) {
    await Task.findOneAndUpdate({ taskId }, { status: 'failed' });
  }
}
```

5.5 llm/nebius.js — LLM Client Setup

Nebius uses an OpenAI-compatible API — which means you can use the openai npm package with a custom baseUrl pointing to Nebius instead of OpenAI.

```
const OpenAI = require('openai');

const nebius = new OpenAI({
  baseUrl: 'https://api.studio.nebius.com/v1/',
  apiKey: process.env.NEBIUS_API_KEY
});

module.exports = nebius;
```

 This is clever: Because Nebius follows the OpenAI API spec, you can swap out the provider by just changing baseUrl and apiKey. Your chains don't care which LLM is behind it.

5.6 chains/gameChain.js — Game Code Generation

A 'chain' is a function that wraps one LLM call with a specific purpose. gameChain.js is responsible for generating the actual game code.

What it does:

- Takes the user's prompt as input
- Imports the gamePrompt template from prompt/gamePrompt.js
- Calls the Nebius LLM via the nebius client
- Tells the model to return a JSON object with { title, html, css, js }
- Parses the LLM response and returns the structured game data

```
const nebius = require('../llm/nebius');
const { getGamePrompt } = require('../prompt/gamePrompt');

exports.generateGame = async (userPrompt) => {
  const { systemPrompt, userMessage } = getGamePrompt(userPrompt);

  const response = await nebius.chat.completions.create({
    model: 'meta-llama/Meta-Llama-3.1-70B-Instruct',
    messages: [
      { role: 'system', content: systemPrompt },
      { role: 'user', content: userMessage }
    ],
    response_format: { type: 'json_object' } // force JSON output
  });

  const result = JSON.parse(response.choices[0].message.content);
  return result; // { title, html, css, js }
};
```

5.7 chains/imageChain.js — Thumbnail Generation

imageChain.js generates a visual thumbnail for the game. It takes the game title and asks the LLM (or an image model) to produce a base64-encoded image.

- Takes the game title as input
- Uses imagePrompt template to craft the image generation prompt
- Calls Nebius image generation API
- Returns a base64 data URI (data:image/png;base64,...) stored in MongoDB

5.8 prompt/gamePrompt.js — The Prompt Template

This is where the prompt engineering lives. The quality of every generated game depends on how well this prompt is crafted.

A well-crafted game prompt system message tells the LLM:

- You are an expert game developer
- Generate a complete, self-contained, playable 2D browser game
- Return ONLY valid JSON — no markdown, no explanation
- The JSON must have exactly these fields: title, html, css, js
- The game must be playable using only keyboard/mouse
- HTML should be a complete document, CSS and JS separate
- Include score tracking, win/lose conditions, and clear instructions

```
// Example structure of gamePrompt.js
exports.getGamePrompt = (userPrompt) => ({
  systemPrompt: `You are an expert browser game developer.
  Your task is to generate a complete, fun, playable 2D browser game.
  Respond ONLY with valid JSON in this exact format:
  { title: string, html: string, css: string, js: string }
  The game must be self-contained and work in a modern browser.` ,
  userMessage: `Create this game: ${userPrompt}`
});
```

 **Prompt Engineering is the real skill here. The LLM is powerful but dumb — it does exactly what the prompt says. A bad prompt gives broken games. A good prompt gives polished, playable experiences.**

5.9 models/Task.js — The MongoDB Schema

This defines the structure of how a game task is stored in MongoDB.

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema({
  taskId: { type: String, required: true, unique: true },
  status: { type: String, enum: ['processing', 'completed', 'failed'],
    default: 'processing' },
  prompt: { type: String },
  title: { type: String },
  html: { type: String },
  css: { type: String },
  js: { type: String },
  thumbnail: { type: String }, // base64 data URI
  createdAt: { type: Date, default: Date.now }
});
```

```
module.exports = mongoose.model('Task', taskSchema);
```

Why store html, css, and js separately instead of one big string? Because:

- Frontend can apply them independently — inject CSS into <style>, JS into <script>
- Easier to update or patch one part without regenerating everything
- Cleaner separation for potential future features (edit just the CSS, etc.)

5.10 middlewares/errorHandler.js — Global Error Handling

Any error passed to next(err) anywhere in the app lands here. One place to handle all errors.

```
module.exports = (err, req, res, next) => {  
  console.error(err.stack);  
  res.status(err.status || 500).json({  
    error: err.message || 'Internal Server Error'  
  });  
};
```

Error handler middleware has FOUR parameters (err, req, res, next) — Express identifies it as an error handler by the four-argument signature.

5.11 public/index.html — The Frontend

A static HTML file served directly by Express (app.use(express.static('public'))). This is the game generator UI that users interact with.

What it does:

- Form for the user to type their game prompt
- Calls POST /api/tasks and gets back a taskId
- Polls GET /api/tasks/:taskId every few seconds
- When status is 'completed', renders the game in an iframe
- Also fetches GET /api/games to show a gallery of previously generated games

5.12 .env.example — Required Environment Variables

```
NEBIUS_API_KEY=your_nebius_api_key_here  
MONGODB_URI=mongodb://localhost:27017/gdg-workshop  
PORT=3000
```

6. How All Folders Are Interconnected

Understanding the dependency graph is key to navigating and extending this project:

index.js	Imports: routes/, middlewares/, llm/ (for DB setup). The root.
routes/	Imports: controllers/. Knows nothing about DB or LLM.
controllers/	Imports: service/. Knows nothing about DB or LLM directly.
service/	Imports: models/, chains/. The orchestrator.
chains/	Imports: llm/, prompt/. Knows about LLM API calls.
llm/	Imports: nothing from project. Just configures the API client.
prompt/	Imports: nothing. Pure functions returning prompt strings.
models/	Imports: mongoose. Defines DB schema.
middlewares/	Imports: nothing. Pure Express middleware.
public/	Imports: nothing. Static frontend, talks to API via fetch()

 *Data flow: request → routes → controllers → service → chains → llm+prompt → back up the chain → MongoDB update → user polls and gets result.*

7. Key Concepts in This Project

7.1 Async Task Queue Pattern

Instead of making the user wait 20-30 seconds for game generation, the backend uses a fire-and-forget pattern:

- Request comes in → save a 'processing' record → return taskId immediately
- Generation happens in background (not blocking the HTTP response)
- Client polls until status changes to 'completed'
- This pattern is used in production by services like Stripe, OpenAI batch API, video processing platforms

7.2 OpenAI-Compatible APIs

Nebius follows the same API spec as OpenAI. This is increasingly common — many LLM providers (Groq, Together AI, Anyscale, Fireworks) use OpenAI-compatible endpoints. The benefit: you use one SDK (openai npm package) and can switch providers by changing just the baseUrl and apiKey.

7.3 Chains — Composable LLM Calls

The chains/ folder represents the idea of chaining LLM calls together. Each chain does one specific thing:

- gameChain — generates playable game code
- imageChain — generates a visual thumbnail

In more complex systems, chains can be sequential (output of one feeds into the next) or parallel. This project runs them sequentially: generate code first, then generate thumbnail for that code.

7.4 Structured Output / JSON Mode

LLMs sometimes return markdown, code fences, or text around the JSON. To prevent this, the project uses response_format: { type: 'json_object' } — this forces the model to return only valid JSON. This is critical for reliable parsing.

7.5 Separation of Prompt from Logic

The prompt/ folder separates prompt templates from the LLM calling code. This is a best practice because:

- Prompts change frequently (you iterate on them to get better output)
- Logic changes infrequently (how you call the API stays the same)
- You can A/B test different prompts without touching chain logic
- Prompts can be version-controlled and reviewed independently

8. Deployment — How It's Deployed on a VM

8.1 Why a VM?

A VPS (Virtual Private Server) gives you full control — no cold starts, persistent connections, custom configs, and it's cheaper long-term for a constantly-running service. For a workshop project, it also makes the demo reliable and always-on.

8.2 The Full VM Deployment Stack

OS	Ubuntu 22.04 LTS — stable, well-documented Linux distro
Node.js	v18+ — installed via NodeSource repository
PM2	Process manager — keeps the app running, auto-restarts on crash
Nginx	Reverse proxy — routes port 80/443 traffic to your Node app on 3000
MongoDB	Either MongoDB Atlas (cloud) or local Mongo on the VM
Environment	.env file on the VM with NEBIUS_API_KEY and MONGODB_URI

8.3 Deployment Commands

1. SSH into the VM

```
ssh root@your-server-ip
```

2. Install Node.js

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

3. Clone the repo

```
git clone https://github.com/sujallchaudhary/gdg-workshop-backend.git  
cd gdg-workshop-backend  
npm install
```

4. Set up environment variables

```
cp .env.example .env  
nano .env    # fill in NEBIUS_API_KEY and MONGODB_URI
```

5. Start with PM2

```
npm install -g pm2
pm2 start index.js --name 'gdg-workshop'
pm2 startup    # auto-start on reboot
pm2 save
```

6. Configure Nginx

```
sudo apt install nginx
sudo nano /etc/nginx/sites-available/gdg-workshop

server {
    listen 80;
    server_name your-server-ip;
    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}

sudo ln -s /etc/nginx/sites-available/gdg-workshop /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl restart nginx
```

✅ After this: visiting `http://your-server-ip` in a browser shows the game generator frontend. Anyone in the workshop can use it to generate their own game!

9. Ideas to Extend This Project

Once you understand the codebase, here are ways to make it more powerful:

Add authentication	Let users sign up, log in, and see only their games. Add JWT middleware.
Add game categories	Let users tag games by genre. Add a category field to the schema.
Add sharing	Generate a unique shareable URL per game. Route that renders just that game.
Stream generation	Use Server-Sent Events (SSE) instead of polling for real-time updates.
Multiple LLM providers	Let users choose between GPT-4, Gemini, Claude, and compare outputs.
Game rating system	Let users upvote or rate games. Add a likes field to the Task model.
Add a game editor	Allow users to edit the generated HTML/CSS/JS in the browser.
Analytics	Track which prompts generate the best games. Store prompt + rating.
Retry failed tasks	Add an endpoint to retry a failed task without re-submitting.